PixnexAI - Complete Project Documentation

Project Overview

PixnexAI is a comprehensive AI-powered SaaS web application that provides multiple image generation and manipulation tools. Built with the MERN stack (MongoDB, Express.js, React, Node.js), it leverages the Clipdrop API to offer professional-grade image processing capabilities.

Live Application: https://pixnex.flickdesk.tech

GitHub Repository: https://github.com/shubhammauryainfo/pixnexAI.git

---

Core Features

1. **Text to Image Generation**

- Convert text descriptions into high-quality images
- AI-powered image synthesis
- Customizable prompts

2. **Cleanup Tool**

- Remove unwanted objects from images
- Requires mask image for precise selection
- Seamless object removal

3. **Image Upscaling**

- Enhance image quality and resolution
- Configurable target dimensions
- AI-based quality enhancement

4. **Remove Background**

- Automatic background removal
- Smart subject detection
- PNG output with transparency

5. **Remove Text**

- Intelligently detect and remove text
- Preserves image quality
- Automatic text detection

6. **Replace Background**

- Replace backgrounds with AI-generated scenes
- Prompt-based background generation

- Natural integration

7. **Uncrop**

- Extend image borders intelligently
- Configurable extension in all directions
- AI-generated seamless extensions

Technology Stack

Frontend

- **React 18.3.1** - Modern UI library
- **Vite 6.0.3** - Fast build tool and dev server
- **TailwindCSS 3.4.17** - Utility-first CSS framework
- **Motion 11.15.0** - Animation library (Framer Motion)
- **React Router DOM 7.1.0** - Client-side routing
- **Axios 1.7.9** - HTTP client
- **React Toastify 11.0.2** - Toast notifications

Backend

- **Node.js** - JavaScript runtime
- **Express 4.21.2** - Web application framework
- **MongoDB 8.9.2** - NoSQL database
- **Mongoose 8.9.2** - MongoDB object modeling
- **JWT (jsonwebtoken 9.0.2)** - Authentication
- **Bcrypt 5.1.1** - Password hashing
- **Axios 1.7.9** - API requests to Clipdrop
- **Form-Data 4.0.1** - Multipart form data
- **CORS 2.8.5** - Cross-origin resource sharing
- **Dotenv 16.4.7** - Environment variables

External APIs

- **Clipdrop API** - AI image processing by Stability AI
- **Razorpay 2.9.5** - Payment gateway (integrated)

System Architecture

Application Flow

```
┌──────────────┐          ┌──────────────┐          ┌──────────────┐
│   Client     │──────────▶│   Server     │──────────▶│  MongoDB     │
│  (React)     │◀──────────│  (Express)   │◀──────────│  Database    │
└──────────────┘          └──────────────┘          └──────────────┘
                                  │
```

```
                              ▼
                        ┌──────────────┐
                        │  Clipdrop    │
                        │    API       │
                        └──────────────┘
```

Authentication Flow

1. User registers/logs in
2. Server validates credentials
3. JWT token generated and sent to client
4. Client stores token in localStorage
5. Token included in all authenticated requests
6. Server validates token via middleware

Credit System

- New users receive 5 free credits
- Each AI operation costs 1 credit
- Credits deducted after successful operation
- Users redirected to purchase page when credits depleted

Project Structure

```
pixnexAI/
│
├── client/                        # Frontend React application
│   ├── public/                    # Static assets
│   ├── src/
│   │   ├── assets/                # Images and icons
│   │   │   └── assets.js          # Asset exports
│   │   ├── components/            # Reusable components
│   │   │   ├── Description.jsx    # Description section
│   │   │   ├── Features.jsx       # Feature cards
│   │   │   ├── Footer.jsx         # Footer component
│   │   │   ├── GenerateBtn.jsx    # Generate button
│   │   │   ├── Header.jsx         # Header/Hero section
│   │   │   ├── Login.jsx          # Login/Signup modal
│   │   │   ├── Navbar.jsx         # Navigation bar
│   │   │   ├── Steps.jsx          # How it works section
│   │   │   └── Testimonials.jsx   # Testimonials section
│   │   ├── context/
│   │   │   └── AppContext.jsx     # Global state management
│   │   ├── pages/
│   │   │   ├── BuyCredit.jsx      # Credit purchase page
│   │   │   ├── Home.jsx           # Landing page
│   │   │   ├── Result.jsx         # Image generation page
│   │   │   └── Tools.jsx          # Image tools page
│   │   ├── App.jsx                # Main app component
```

```
│   │   ├── main.jsx                # Entry point
│   │   └── index.css               # Global styles
│   ├── .env                        # Environment variables
│   ├── package.json                # Dependencies
│   ├── vite.config.js              # Vite configuration
│   └── tailwind.config.js          # Tailwind configuration
│
├── server/                         # Backend Node.js application
│   ├── config/
│   │   └── mongodb.js              # Database connection
│   ├── controllers/
│   │   ├── imageController.js      # Image processing logic
│   │   └── userController.js       # User management logic
│   ├── middlewares/
│   │   └── auth.js                 # JWT authentication
│   ├── models/
│   │   └── userModel.js            # User schema
│   ├── routes/
│   │   ├── imageRoutes.js          # Image API routes
│   │   └── userRoutes.js           # User API routes
│   ├── .env                        # Environment variables
│   ├── package.json                # Dependencies
│   ├── server.js                   # Server entry point
│   └── vercel.json                 # Vercel deployment config
│
└── README.md                       # Project readme
```

Database Schema

User Model

```jsx
{
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  password: {
    type: String,
    required: true  // Hashed with bcrypt
  },
  creditBalance: {
    type: Number,
    default: 5
```

```
    }
  }
```

API Endpoints

User Routes (`/api/user`)

Method	Endpoint	Auth	Description	Request Body	Response
POST	`/register`	No	Register new user	`{ name, email, password }`	`{ success, token, user }`
POST	`/login`	No	User login	`{ email, password }`	`{ success, token, user }`
GET	`/credits`	Yes	Get user credits	-	`{ success, credits, user }`

Image Routes (`/api/image`)

Method	Endpoint	Auth	Description	Request Body	Response
POST	`/generate-image`	Yes	Generate image from text	`{ prompt }`	`{ success, message, creditBalance, resultImage }`
POST	`/cleanup`	Yes	Remove objects	`{ imageFile, maskFile }`	`{ success, message, creditBalance, resultImage }`
POST	`/upscale`	Yes	Upscale image	`{ imageFile, targetWidth, targetHeight }`	`{ success, message, creditBalance, resultImage }`
POST	`/remove-background`	Yes	Remove background	`{ imageFile }`	`{ success, message, creditBalance, resultImage }`
POST	`/remove-text`	Yes	Remove text	`{ imageFile }`	`{ success, message, creditBalance, resultImage }`

Method	Endpoint	Auth	Description	Request Body	Response
POST	`/replace-background`	Yes	Replace background	`{ imageFile, prompt }`	`{ success, message, creditBalance, resultImage }`
POST	`/uncrop`	Yes	Extend image	`{ imageFile, extendLeft, extendRight, extendUp, extendDown }`	`{ success, message, creditBalance, resultImage }`

Auth Header Format: `{ "token": "jwt-token-here" }`

Key Implementation Details

Backend Controllers

Credit Management

```
// Helper function to check user credits
const checkUserCredits = async (userId) => {
  const user = await userModel.findById(userId);
  if (!user) {
    return { success: false, message: "User not found" };
  }
  if (user.creditBalance === 0 || user.creditBalance < 0) {
    return {
      success: false,
      message: "No credit balance",
      creditBalance: user.creditBalance,
    };
  }
  return { success: true, user };
};

// Helper function to deduct credits
const deductCredits = async (userId, currentBalance) => {
  await userModel.findByIdAndUpdate(userId, {
    creditBalance: currentBalance - 1,
  });
  return currentBalance - 1;
};
```

Image Processing Flow

1. Validate user authentication
2. Check credit balance

3. Prepare FormData for Clipdrop API
4. Convert image to base64 (frontend to backend)
5. Send request to Clipdrop API
6. Receive binary image data
7. Convert to base64 string
8. Deduct credit from user account
9. Return result to frontend

Frontend Context Management

The `AppContext` provides global state management including:

- User authentication state
- Credit balance
- Token management
- API functions for all tools
- Login modal visibility

Authentication Middleware

```
const userAuth = async (req, res, next) => {
  const { token } = req.headers;
  if (!token) {
    return res.json({
      success: false,
      message: "Not authorized. Login again.",
    });
  }
  try {
    const tokenDecode = jwt.verify(token, process.env.JWT_SECRET);
    if (tokenDecode.id) {
      req.body.userId = tokenDecode.id;
    } else {
      res.json({ success: false, message: "Not authorized. Login again." });
    }
    next();
  } catch (error) {
    res.json({ success: false, message: error.message });
  }
};
```

Environment Variables

Server (.env)

```
MONGODB_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret_key
```

```
CLIPDROP_API=your_clipdrop_api_key
PORT=5000
```

Client (.env)

```
VITE_BACKEND_URL=http://localhost:5000
```

Installation and Setup

Prerequisites

- Node.js (v16 or higher)
- MongoDB database
- Clipdrop API key

Server Setup

```
# Navigate to server directory
cd server

# Install dependencies
npm install

# Create .env file with required variables
# MONGODB_URI, JWT_SECRET, CLIPDROP_API, PORT

# Start server
npm start
```

Server will run on `http://localhost:5000`

Client Setup

```
# Navigate to client directory
cd client

# Install dependencies
npm install

# Create .env file
# VITE_BACKEND_URL=http://localhost:5000

# Start development server
npm run dev
```

Client will run on `http://localhost:5173`

User Flow

Registration/Login Flow

1. User clicks "Get Started" or "Login"
2. Login modal appears
3. User enters credentials
4. Form submits to `/api/user/register` or `/api/user/login`
5. Server validates and returns JWT token
6. Token stored in localStorage
7. User data and credits loaded
8. Modal closes

Image Generation Flow

1. User navigates to "Result" page
2. Enters text prompt
3. Clicks "Generate"
4. Frontend calls `generateImage()` from context
5. Request sent to `/api/image/generate-image`
6. Server checks credits and processes
7. Clipdrop API generates image
8. Credit deducted
9. Base64 image returned
10. Image displayed with download option

Image Tools Flow

1. User clicks feature card on home page
2. Redirected to Tools page with tool pre-selected
3. User uploads required image(s)
4. Enters additional parameters (if needed)
5. Clicks "Process Image"
6. Frontend calls appropriate tool function
7. Server processes with Clipdrop API
8. Credit deducted
9. Result displayed with download option

Security Features

Password Security

- Passwords hashed using bcrypt (10 salt rounds)
- Never stored in plain text
- Secure comparison during login

JWT Authentication

- Token-based authentication
- Tokens stored in localStorage
- Middleware validates all protected routes
- Token contains user ID only

API Security

- CORS enabled for cross-origin requests
- Environment variables for sensitive data
- Authentication required for all image operations

Error Handling

Backend

- Try-catch blocks in all controllers
- Meaningful error messages returned
- Console logging for debugging
- Credit balance validation before operations

Frontend

- Toast notifications for user feedback
- Error messages displayed clearly
- Automatic redirect to buy credits page
- Form validation before submission

Clipdrop API Integration

Supported Endpoints

1. **Text to Image:** `https://clipdrop-api.co/text-to-image/v1`
2. **Cleanup:** `https://clipdrop-api.co/cleanup/v1`
3. **Upscale:** `https://clipdrop-api.co/image-upscaling/v1/upscale`
4. **Remove Background:** `https://clipdrop-api.co/remove-background/v1`
5. **Remove Text:** `https://clipdrop-api.co/remove-text/v1`
6. **Replace Background:** `https://clipdrop-api.co/replace-background/v1`
7. **Uncrop:** `https://clipdrop-api.co/uncrop/v1`

Request Format

- FormData with image files
- API key in headers: `x-api-key`
- Response as arraybuffer
- Conversion to base64 for frontend

UI/UX Features

Design Elements

- Gradient backgrounds (teal to orange)
- Smooth animations using Motion library
- Responsive design (mobile, tablet, desktop)
- Card-based feature showcase
- Modal-based login system

User Experience

- Loading indicators during processing
- Progress bars for image generation
- Immediate visual feedback
- Toast notifications for success/error
- Clear credit balance display
- Easy image download functionality

Future Enhancements

Planned Features

1. Complete Razorpay payment integration
2. Image history/gallery for users
3. Batch processing capabilities
4. Advanced image editing options
5. Social sharing functionality
6. Admin dashboard for user management
7. Email verification system
8. Social login (Google, GitHub)
9. Image favorites/bookmarks
10. API rate limiting

Potential Improvements

- Image caching for faster loads
- Compression before upload
- Progress tracking for long operations
- Real-time collaboration features
- Mobile app version
- Advanced prompt suggestions
- Image style presets

Deployment

Hosting Platforms

- **Frontend:** Vercel/Netlify
- **Backend:** Vercel/Heroku/Railway
- **Database:** MongoDB Atlas

Production Considerations

- Environment variables configured
- CORS settings updated for production URL
- MongoDB connection string updated
- API keys secured
- Build optimization enabled
- Error logging implemented

Performance Optimization

Frontend

- Code splitting with React Router
- Lazy loading of components
- Optimized images and assets
- Vite's fast build system
- Minimal bundle size

Backend

- Efficient database queries
- Connection pooling with MongoDB
- Async/await for non-blocking operations
- Error handling to prevent crashes

Testing

Manual Testing Checklist

- ☑ User registration
- ☑ User login
- ☑ Token persistence
- ☑ Credit balance updates
- ☑ Image generation
- ☑ All 6 image tools
- ☑ File upload functionality
- ☑ Download functionality
- ☑ Responsive design
- ☑ Error handling

Troubleshooting

Common Issues

Server not starting:

- Check MongoDB connection string
- Verify all environment variables
- Check port availability

404 errors on API calls:

- Restart server after code changes
- Verify routes are registered
- Check VITE_BACKEND_URL in client

Image not displaying:

- Check Clipdrop API key validity
- Verify image format (base64)
- Check network tab for errors

Credit not deducting:

- Verify MongoDB connection
- Check user authentication
- Review server logs

Credits and Acknowledgments

Technologies Used

- MERN Stack (MongoDB, Express, React, Node.js)
- Clipdrop API by Stability AI
- TailwindCSS for styling
- Motion for animations

Developer

- **GitHub:** shubhammauryainfo

License

This project is developed as part of a third-year project. All rights reserved.

Contact

For questions, suggestions, or support:

- **Live Demo:** https://pixnex.flickdesk.tech
- **GitHub:** https://github.com/shubhammauryainfo/pixnexAI

Last Updated: February 5, 2026